

Causal Inference for HPC Reliability: Methods and Insights from Real-World Workload Datasets

Armaan Ashfaque
aashf@uic.edu

University of Illinois at Chicago
Chicago, Illinois, USA

Smita Darmora
sdarm@uic.edu

University of Illinois at Chicago
Chicago, Illinois, USA

Sarah Zahir Syeda
ssyed96@uic.edu

University of Illinois at Chicago
Chicago, Illinois, USA

Abstract

Job failures in High Performance Computing (HPC) are a serious concern, however current research has largely established correlations between failure rates and the factors influencing them without identifying the underlying causal structure. To investigate potential causes of task failure, we apply modern causal discovery methods to datasets from MIT Supercloud and ALCF Theta. We use PC, FCI, and GES on the MIT logs and NOTEARS and FCI on the Theta logs under both continuous and discretized representations. For MIT, baseline models often showed Job Status with few or no direct causal parents, but more aggressive binning produced a more stable and interpretable DAG with meaningful edges into Status, supported by a graph falsification test. Using this refined graph, interventional analysis suggests a modest increase in failure risk with higher GPU usage. For Theta, we observed associations such as specified walltime with `exit_fail`, though directions remained unresolved. Overall, our results show that careful preprocessing and validation are key to extracting actionable causal insights from HPC logs.

Our Github repository can be found at [Code](#)

1 Introduction

High performance computing environments waste resources due to job failures, which is a significant concern for operational efficiency and research outcomes. Past studies [1] [2] examining job logs from major supercomputing environments such as Tachyon [3] and SURF Lisa [4] show that larger jobs requiring more nodes, cores and memory tend to fail more frequently. However, these findings are mainly correlational and do not confirm causal relationships between the resource metrics and failure probability. The problem is further complicated by the presence of mixed—discrete and continuous—variables and potentially nonlinear, non-Gaussian system dynamics that can obscure the true structure of dependencies.

In this paper, we address this gap by applying modern causal discovery techniques to real-world datasets from MIT Supercloud [5] and ALCF Theta [6]. Our goal is to move beyond descriptive associations and enable counterfactual questions such as: would the failure probability change if the same job had requested fewer resources? Answering such questions can provide more actionable insight into how job configuration choices influence reliability and may help explain why size-related correlations are not always consistent across systems.

Our methodology follows a two-stage structure aligned with our project plan: (1) causal structure discovery and (2) causal effect estimation. We begin with exploratory data analysis to guide feature selection and preprocessing decisions, focusing on scheduler-level variables related to resource allocation and execution time. Because

causal discovery methods often assume either discrete or continuous inputs, we evaluate both representations to understand how data type choices affect graph stability and interpretability. This design allows us to examine whether the observed lack of direct causal parents for failure outcomes is due to genuine system complexity, limited feature observability, or sensitivity to modeling assumptions.

Causal modeling has shown promise in related large-scale systems contexts. While WANDER [7] offers decision support through counterfactual analysis to maximize user objectives under constraints, CausalInfer [8] supports performance diagnosis in distributed systems by building hierarchical causality graphs to identify root causes. Motivated by these directions, our study aims to construct an interpretable causal map of HPC job dynamics using constraint-based, score-based, and optimization-based discovery methods. By applying PC, FCI, and GES on the MIT Supercloud logs and NOTEARS and FCI on the ALCF Theta logs, we seek to clarify what causal claims can be supported by current operational logs and to lay groundwork for more causally informed analysis of job failure behavior.

2 Related Works

Our study expands on previous research in the three key areas mentioned below, providing deeper insight into how causal discovery methods, operational datasets, and effect-estimation techniques interact.

2.1 Prior work on HPC operational datasets.

Large-scale systems, such as the MIT Supercloud and ALCF Theta [5, 9], are highly dependent on operational stability and efficiency, especially as modern AI-driven workloads continue to increase both scale and complexity. Previous research utilizing these datasets frequently concentrates on performance characterization and predictive modeling, including workload analysis [5] and I/O monitoring [9]. However, relying on correlation is a major drawback that hinders the discovery of the underlying causes required to improve task dependability and operational efficiency, as demonstrated by recent research on handling massive data from HPC systems [1]. Our work aims to tackle this by using these two different HPC datasets—one on job-level metrics (MIT Supercloud) and the other on system-wide performance (ALCF Theta)—in order to identify the actual structural dependencies that control system behavior and possible failures. Developing proactive, causally aware resource management methods requires a shift from correlational observation to causal mapping, particularly as AI-oriented HPC workloads continue to evolve and introduce new forms of nonlinear behavior.

2.2 Overview of Different Causal Discovery Algorithms.

To ensure reliable discovery across diverse data types, we use algorithms from three different paradigms: constraint-based, score-based, and optimization-based. The Markov Condition and Faithfulness principles serve as the foundation for Constraint-Based Methods (PC and FCI) [10]. To create a Completed Partially Directed Acyclic Graph (CPDAG), the PC algorithm iteratively checks for conditional independencies to prune edges [10]. The FCI algorithm [11] expands PC by introducing rules to take into account the presence of unobserved confounding variables (UOCs), and produces a Partial Ancestral Graph (PAG). For Score-based methods, we talk about the Greedy Equivalence Search (GES) algorithm, which is a structure learning method that finds the structure that maximizes a penalized likelihood score by searching the space of Markov equivalency classes [10]. Goodness-of-fit and model complexity are balanced by popular metrics like the Bayesian Information Criterion (BIC) for continuous data and the BDeu (Bayesian Dirichlet equivalent uniform) score for discrete data. [10].

Some datasets are characterized by nonlinearity and non-Gaussian distributions, which require the use of advanced Functional Causal Models (FCMs) that overcome the limitations of traditional linear and Gaussian-based methods. These models include LiNGAM, PNL, ANM and optimization-based methods according to the previous literature. In [12], the paper talks about LiNGAM (Linear Non-Gaussian Acyclic Model) which uniquely identifies the causal direction in linear systems, by taking advantage of the non-Gaussianity of noise factors, frequently employing Independent Component Analysis (ICA). Another paper [13], talks about the ANM (Additive Noise Model) which addresses complex functional relationships and the PNL model (where the relationship is $Y = g(f(X) + \epsilon)$, with g being an outer invertible nonlinearity) which particularly provides enhanced identifiability in situations where nonlinearity is also introduced by measurement or observation methods. In addition, we also look at optimization-based methods. More recent techniques, such as NOTEARS [14], employ continuous optimization to enforce the directed acyclic graph (DAG) constraint using a differentiable penalty function in order to scale causal discovery to high-dimensional and non-linear data.

Additionally, after having the DAGs produced using the appropriate algorithms, the reliability of these inferred structures, especially in complex, real-world systems like HPC environments, is often questionable due to violations of underlying assumptions. Hence, quantitatively evaluating the consistency of a graph is of utmost importance. In order to do this, the 2024 paper [15], proposed an approach to falsify causal discovery algorithms without ground truth by determining the association between two temporarily excluded variables (X and Y), known as the Leave-One-Variable-Out (LOVO) prediction. Simulations demonstrate the efficiency of the strategy by confirming a high correlation between the overall correctness of the learnt causal structure and the resulting prediction error. More works [16] have leveraged falsifying causal graphs by permutation based tests. They introduced a novel metric which aims to clarify whether a user-specified graph's violations are false positives or indicate actual flaws in the graph by comparing it against a baseline that is constructed by randomly permuting its nodes.

2.3 Overview of Causal Effect Estimation Techniques

Modeling the causal relationships between components is essential for understanding and analyzing the system. Effect estimation problems, where the objective is to quantify the impact of an intervention of one variable on another, are one of the common uses of causality [17]. Causal questions frequently extend beyond effect estimation problems. [18, 19]

Causal effect estimation is responsible for quantification of the impact of a given treatment T on an outcome Y from the observation data. The aim is to estimate the average causal effect (ACE), $E[Y(1) - Y(0)]$, where $Y(1)$ and $Y(0)$ are the unobserved potential outcomes. This estimation is based on the definition of causal assumptions and the identifiability of the ACE which transforms the unobservable counterfactual quantity into an observed one. Several statistical methods are employed such as regression based methods, propensity score methods, instrumental variables and Quasi-experimental designs [20].

Some of the propensity score methods include propensity score matching, which matches treated units to control units with similar propensity score. On the other hand, there is inverse probability weighting which weighs each observation by the inverse of its propensity score, allowing for weighted ATE calculation [21]. According to [22], instrumental variables are used when there are unobserved confounders. They are correlated with the treatment, affect the outcome only through its effect on the treatment and is independent of any unobserved confounding. The Quasi experimental designs, often described as nonrandomized, pre-post intervention studies [23] leverage models like Difference-in-Differences (DiD) and Regression Discontinuity Design (RDD). DiD compares the changes in the outcome over time for a treated group with change for a control group assuming parallel trends in the absence of treatment whereas, RDD estimates the causal effect for individuals who are near a cut off point used to assign treatment [24].

Successful application of causal inference requires establishing assumptions about the mechanisms behind observed data and determining if and to what extent they are correct, in addition to effective statistical estimators of a treatment's effect [25]. Some of the software packages that exist for undertaking causal inference and structure discovery include, causaleffect, dagitty, dosearch, pcalg, bnlearn and more [26]. However, the majority of causal inference libraries primarily concentrate on offering strong statistical estimators. Hence, [25] presents DoWhy, an open-source Python library based on the formal framework of causal graphs for defining and testing causal assumptions. DoWhy was developed using causal assumptions as its first-class inhabitants. An API for the four phases that are typical of every causal analysis is provided by DoWhy.

3 Formal Problem Description

The main goal of this work is to identify the factors that affect job execution status in HPC settings. Guided by our literature review and Exploratory Data Analysis (EDA), we aim to examine the causal relationship between a job's descriptive scheduler metadata and its success rate across two real-world systems: MIT Supercloud and ALCF Theta. Rather than treating "job size" as a single

variable, we model it as a multi-dimensional construct composed of resource allocation parameters such as assigned nodes, CPUs, memory, and GPUs. This choice is consistent with our preliminary analysis, which suggests that these resource attributes, together with execution-time variables, are strongly interrelated and may influence job outcomes in combination rather than in isolation. To better capture the complexity of these interactions, our formulation intentionally avoids oversimplified assumptions and instead reflects the heterogeneous nature of workloads observed across both systems.

Accordingly, our study focuses on learning a structured causal representation of these interactions and using it to reason about how changes in individual resource or time-related factors could shift the likelihood of successful completion. This framing provides a more nuanced view of job reliability than approaches that rely on a single proxy (e.g., node count) to represent workload scale. In our analysis, *Status* denotes the job completion outcome variable for the MIT Supercloud dataset, while *EXIT_FAIL* denotes the corresponding failure indicator for the ALCF Theta dataset. This consistent labeling allows us to compare outcomes across systems while still respecting the differences in how each platform records job termination details.

4 Proposed Causal Analysis Pipeline

We adopt a two-stage causal analysis pipeline to move beyond correlational patterns and investigate which job- and system-level factors may influence execution outcomes. The pipeline is designed for scheduler logs that contain a mixture of discrete and continuous variables and includes both structure learning and effect estimation steps, allowing us to systematically disentangle complex dependencies that may otherwise remain obscured in large-scale operational data. This section describes our approach at a method level; dataset-specific variables and cleaning details are described later, along with considerations that guided our preprocessing choices and overall analysis workflow.

Stage 1: Causal Structure Discovery

We begin with EDA-guided feature selection focusing on resource allocation and time-related metadata, and we incorporate a small set of derived workload indicators to better represent job scale and operating context. Since causal discovery algorithms typically assume either discrete or continuous inputs, we compare two parallel preprocessing strategies to understand how representation affects the learned structure. In the *Discrete* strategy, we apply simple transformations to reduce skew in heavy-tailed variables and then discretize selected features into bins. In the *Continuous* strategy, we retain the numeric values and just standardize the feature values. It is common for causal discovery algorithms to have varying performance across datasets, with some being better suited to certain context. Due to this, we apply several algorithms-PC[10], FCI[11], GES[10] and NOTEARS[14]-under both strategies by choosing appropriate conditional independence (CI) tests and score functions. To improve clarity around the outcome structure, we also evaluate more aggressive binning configurations. In addition, we apply a graph falsification test [27] to check whether a candidate DAG is broadly consistent with dependencies observed in the data. When local graph diagnostics suggest that a specific variable contributes

weakly or introduces instability, we remove it and re-run discovery before selecting a final structure for downstream analysis.

Stage 2: Causal Effect Estimation

After selecting the most interpretable and supported structure from Stage 1, we estimate interventional effects using a graphical causal model. The DoWhy library allows you to define a DAG and fit causal mechanism to it based on observational data. This enables intervention queries of the form $do(X = x)$ with the job outcome as the target. Because our best-performing discovery setup relies on discretized representations, we estimate effects by comparing meaningful categories of resource usage and time-related variables. This provides adequate support for average causal effect estimation from observational logs and offers an interpretable way to assess how changes in key workload characteristics may shift failure risk.

Together, these two stages provide a practical and reproducible approach for learning plausible causal structure from mixed-type scheduler data and for quantifying outcome changes under targeted interventions.

5 Data description and cleaning

5.1 MIT Supercloud Dataset

The MIT Supercloud dataset[5] is a rich collection of parsed logs from the MIT Supercloud TX-Gaia cluster [28] that spans over 10 months of activity. The data released includes collections from the scheduler, filesystem, compute nodes, CPU and GPU usage. For our work we will stick to just the scheduler logs. This scheduler data uses Slurm [29] to schedule workloads. The scheduler records metadata for every job that has been scheduled over a collection period. The fields recorded provide information on resources requested/allocated, execution status, job submission, wait and runtimes.

5.1.1 Exploratory Data Analysis (EDA): The MIT SuperCloud analysis confirms the data’s high heterogeneity, nonlinearity, and non-Gaussianity, with difference heatmaps revealing that strong correlations are monotonic but non-linear, following curved rather than straight trends (Fig 1.). The non-normal nature is confirmed by Q-Q plots for nearly all variables, particularly time and resource counts, which exhibit heavy-tailed distributions with skewness and extreme outliers that curve dramatically away from the reference line (Fig 2.). Consequently, these findings collectively demonstrate that simple linear additive models cannot sufficiently explain the system’s behavior and suggest a complex, heterogeneous underlying structure driven by diverse and variable workloads.

5.1.2 Feature Selection and Cleaning for Causal Discovery: In order to prepare the dataset for Causal Discovery, derived metrics such as *node_hours* (capturing size and duration), *user_job_frequency* (user experience proxy), and *system_load* (concurrent job count and system stress) along with available features *nodes_alloc*, *cpus_alloc*, *mem_alloc*, *runtime*, *status*, *num_alloc_gpus* were chosen based on EDA and intuition regarding "job size" contribution. Since CD algorithms typically require either purely discrete or continuous variables, we tested both strategies: the Discrete strategy first applied a $\log(1+x)$ transformation to skewed continuous features like runtime before discretizing them into equal frequency bins using

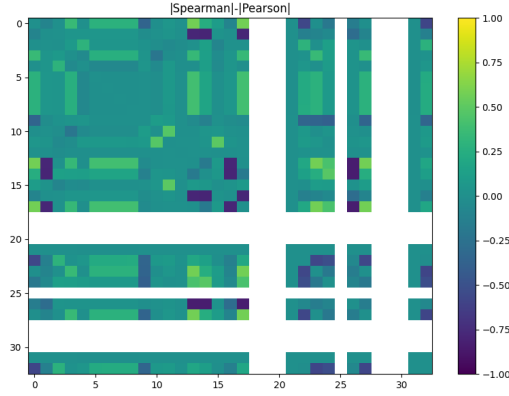


Figure 1: Gaussian Linearity

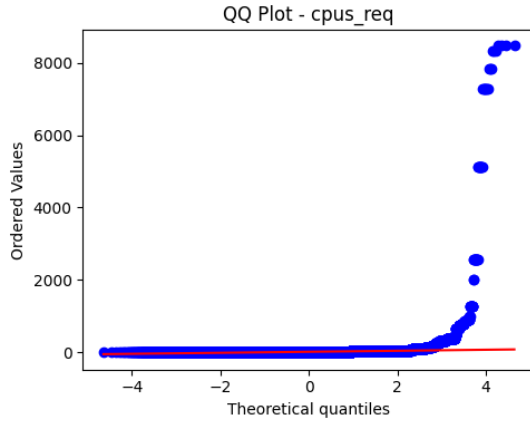


Figure 2: MIT Q-Q Plot

`pandas.qcut()`. Conversely, the Continuous strategy retained the features as they were, applying only `StandardScaler` to all non-binary features before analysis.

5.2 ALCF Theta

We analyze seven years of scheduler logs from the ALCF Theta supercomputer (2017–2023). Theta is scheduled by Cobalt, and we use the Cobalt job table (often referred to as `DIM_JOB_COMPOSITE`) as our primary source. Each record includes the COBALT job ID (`COBALT_JOBID`), user/project, queue, requested resources, submission/start/end timestamps, and exit status. In addition, we reference the Theta hardware-error stream for future steps (we do not use it in the current results but keep it ready for causal tests). Because Theta logs contain many distinct exit status codes (e.g., 0, 1, 121, and others), for the purpose of our causal analysis we consolidate them into a binary outcome, treating exit code 0 as success and any non-zero code as a failure indicator.

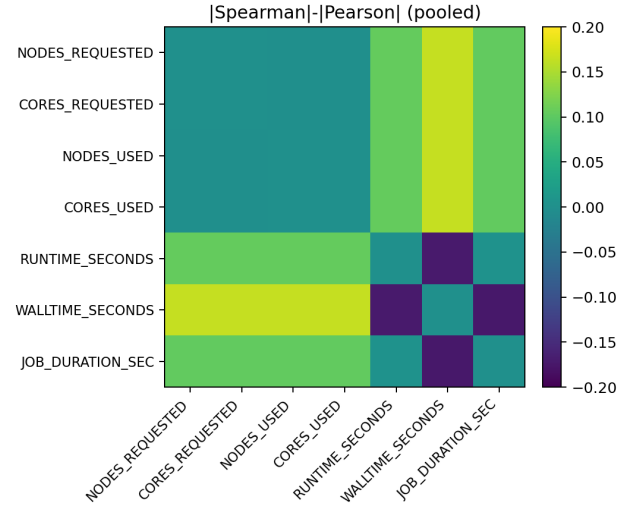


Figure 3: Theta Correlation

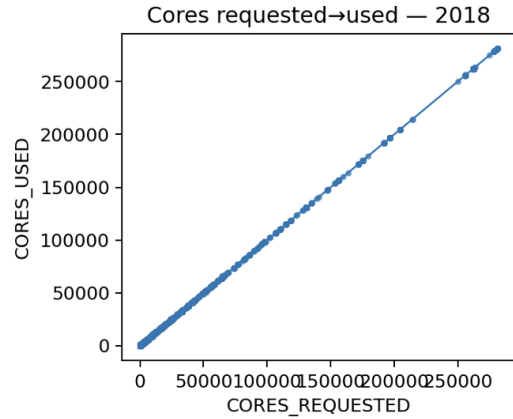


Figure 4: Theta Q-Q plots

5.2.1 Exploratory Data Analysis (EDA): The ALCF Theta data’s primary resource and time metrics display strong correlations, particularly the near-linear connections and high proportionality between requested and used resources (cores and nodes). Although correlation heatmaps (Fig 3.) show consistent blocks, the relationships involving execution time are revealed to be monotonic but non-linear, suggesting that simple linear modeling is insufficient across all operational regimes. This is supported by Q-Q plots (Fig 4.) confirming that time-based variables are log-normally distributed with heavy tails and a rightward skew, which indicates the system’s generative mechanisms are predominantly multiplicative rather than additive, thus leading to a stable causal structure that demands models beyond the simple linear additive framework.

5.2.2 Feature Selection and Cleaning for Causal Discovery: From the raw Cobalt job logs, which were first concatenated into

a single harmonized table, we selected key fields including job identifiers, USER, PROJECT, queue/partition, submission/start/end timestamps, resource counts (e.g., nodes), and EXIT_STATUS. We constructed derived time-based features: `runtime_seconds`, which captures the total time a job spent running by taking the difference between its end time and start time, and `wait_seconds`, which represents the time a job remained in the queue before execution by measuring the interval between its submission time and start time. Our binary outcome variable is `exit_success` (1 if `EXIT_STATUS` == 0, else 0). For data cleaning, we retained only these analysis fields; crucially, we also prepared a separate hardware error stream by collapsing repeated errors on the same failed component within a ≈ 70 -second window. The current Causal Discovery investigation utilizes only the processed job logs.

6 Experimental Setup and Results

This section summarizes our experimental design and key findings on both datasets. Following our proposed pipeline, we focus on two goals: (1) evaluating how different causal discovery (CD) methods behave under mixed data representations, and (2) assessing whether the most interpretable discovered structure can support downstream causal effect estimation.

6.1 MIT Supercloud

Experimental Setup. We applied three causal discovery algorithms—PC, FCI, and GES—to the MIT Supercloud features selected through EDA. Since the data contains a mix of discrete and continuous variables, we used the dual-preprocessing strategy described in Section 4. In our initial round of experiments we chose the G-squared (G2) CI test and the BDeu score for the *Discrete* strategy, and Fisher’s Z test along with the BIC score for the *Continuous* strategy. However, these choices yielded underwhelming results. Following this, we pivoted to using the Pillai trace CI test and BIC-CG score for both strategies, as they were known to perform well on mixed data. We ran PC, FCI, and GES under both strategies and compared the learned graphs, with a focus on the *Status* outcome (success/failure).

Results: Across many baseline runs (PC, FCI, GES), the learned graphs did not show clear direct causes leading into *Status*. This suggests either that important failure drivers are not fully captured in the available features or that the relationship between our observed variables and failure is not easily captured under these discovery settings. The most consistent and intuitive edges were observed among non-status variables. For example, `node_hours` was reliably linked to `runtime` and `nodes allocated`, and system-level context features showed expected relationships with allocation variables as seen in Figure 5.

To improve interpretability around *Status*, we constrain *Status* as a sink node and use an aggressive discretization strategy. This involves binning all variables into four or fewer bins. While this approach reduces resolution, it mitigates issues arising from variables with widely extreme variation in their values. Under this approach, we obtained a more stable DAG using GES in which *Status* had meaningful incoming edges (See Fig. 7) and no outgoing edges.

We then applied a graph falsification test proposed by Eulig et al.[27] to evaluate whether the discovered structure was broadly

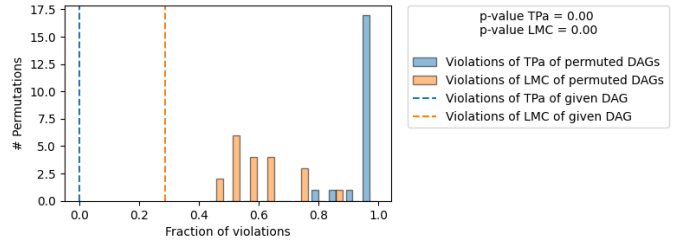


Figure 5: Falsification Report

consistent with our data. The initial falsification report identified the partition variable as problematic. We therefore omitted this variable and re-ran the GES discovery algorithm, producing the DAG shown in Figure 7. A subsequent falsification test on this revised graph revealed no issues, as illustrated in Figure 5. The final falsification report indicates that the learned DAG is informative because 0/20 of the permutations falls within its Markov equivalence class and it violates 4/14 LMCs and is better than 100.0% of the permuted DAGs. These results suggest that the graph is acceptable for downstream causal analysis.

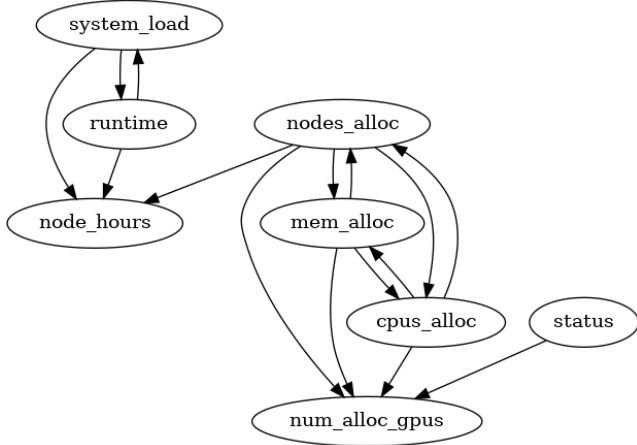
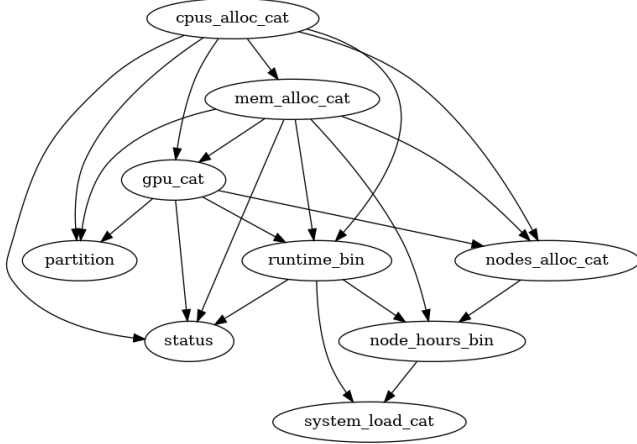
Interventional Analysis: Using the refined MIT causal structure, we fit a graphical causal model and estimate the Average Causal Effect (ACE) of interventions with *Status* as the outcome. Because the discovery pipeline relied on discretized features—where each variable was grouped into three or four bins (four in the case of the `nodes_allocated` variable)—we computed ACE values for category-level interventions (e.g., no GPU vs. single GPU vs. multi-GPU; short vs. long runtime).

This discretized setup allows for more interpretable intervention comparisons. Working directly with the raw continuous values would have produced a large number of possible intervention configurations (e.g., many runtime values or resource allocation levels), making ACE comparisons far more tedious and difficult to interpret in practice. We focus on comparing interventions on between the larger resource categories and the smallest for the parents of *status*. Specifically, we evaluate effects of the form $P(Y=Status|do(x=medium/large)) - P(Y=Status|do(x=small))$, to assess how increasing resource levels influences job success. From the table we can see that the actual causal effect values aren’t really big.

The results are reported in Table 1. Note that *Status* is a binary variable, where 1 indicates job failure and 0 indicates job success. Thus, positive ACE values imply that increasing a resource allocation raises the probability of job failure, while negative ACE values indicate that increasing the resource reduces failure risk. Although the reported ACE values are not large enough to draw definitive conclusions, some trends can be observed. Increasing GPU and CPU allocations appears to have a negative impact on job success, potentially increasing the likelihood of failure. In contrast, higher memory allocation seems to improve job success, which can be explained by the reduced risk of running out of memory. Lastly, longer runtimes are associated with better job outcomes, possibly

Table 1: Average Causal Effect (ACE) estimates for different interventions on the parents of *Status*

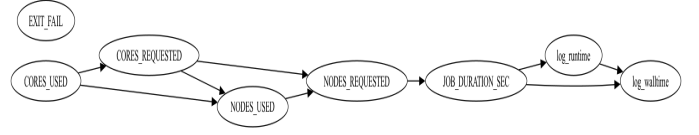
Variable	Large-Small	Medium-Small
gpu_cat	0.0042	0.0003
runtime_bin_cat	-0.0026	-0.0008
cpus_alloc_cat	0.0023	0.0042
mem_alloc_cat	-0.0035	0.0001

**Figure 6: PC-Stable — MIT data****Figure 7: GES— MIT data**

because user’s deploy many short running debugging runs, which is reflected in the observed effect.

6.2 ALCF Theta

Experimental Setup: For the Theta dataset, we used NOTEARS as the primary scalable causal discovery method. We ran a linear NOTEARS model and constrained the binary outcome *EXIT_FAIL* (0/1) to be a sink node for interpretation. This design was used to reduce ambiguity in the learned structure around the failure outcome

**Figure 8: Theta (2018) — NOTEARS DAG with *EXIT_FAIL* forced as a sink**

and to ensure that all inferred edges represent potential causal influence rather than artifacts of model orientation. In addition, we verified that the optimization procedure converged consistently across multiple runs, helping confirm that the learned structure was not overly sensitive to initialization or noise in the dataset. To check robustness to possible unobserved factors, we additionally ran a small-scale FCI experiment on a 10k-job slice of the Theta logs and examined the resulting PAG with *EXIT_FAIL* treated as the major sink outcome. This subsampled analysis allowed us to test whether hidden confounding might distort causal directions in the full dataset, providing a complementary perspective to the linear NOTEARS model.

Results: The NOTEARS model did not identify strong, direct causes of *EXIT_FAIL*. This suggests that a simple linear structure may not be sufficient to explain job failure using the available Theta features. However, NOTEARS recovered an intuitive timing pattern among non-outcome variables: requested/used resources flow into job duration, which then relates to runtime and walltime. This confirms that the job execution timing structure is captured well by the model and that NOTEARS is still able to extract meaningful operational relationships even when the targeted failure signal remains weak. The FCI analysis provided an additional view of relationships when unobserved factors may exist. In particular, it identified a clear association between *EXIT_FAIL* and specified walltime, though the direction was not resolved. This observation connects user-defined time limits to failure behavior and motivates further investigation with models better suited to non-linear structure and binary outcomes. Combined, the NOTEARS and FCI results indicate that Theta’s failure dynamics likely arise from a combination of subtle operational influences and potential unobserved factors, encouraging the use of richer feature sets or hybrid causal discovery strategies in future work.

7 Discussion and Ideas for Next Steps

Our main takeaway is that extracting reliable causal signals about job failure from operational HPC logs is possible, but strongly depends on representation choices and feature coverage. In the MIT analysis, most baseline discovery runs did not produce clear direct parents of the *Status* outcome, suggesting that either key failure drivers are not fully captured in the available metadata or that the signal is sensitive to how mixed variables are handled. However, when we applied more aggressive discretization, we obtained a more interpretable structure with meaningful edges into *Status*. This improvement, together with the graph falsification results, indicates that careful binning can be a practical way to stabilize causal discovery and make downstream analyses more actionable. Using the refined graph, our interventional results hint that higher

GPU and CPU allocations corresponds to an increase in failure risk, while longer runtimes and greater memory allocation show a slight decrease in observed failure.

A second important observation is that the two systems we studied differ enough that cross-dataset causal transfer is not straightforward. During our discussions, we considered mapping the MIT DAG onto the Theta feature space to re-run the same interventional analysis. We ultimately did not pursue this because the overlap between variables is limited, and key resource dimensions that appear important in the MIT results (notably GPU-related features) are not available in the Theta data we used. Given this mismatch, directly reusing the MIT graph could introduce assumptions that are not supported by the Theta logs. This reinforces the idea that causal graphs derived from scheduler data are system-specific and strongly tied to what each platform logs and exposes.

There are several natural next steps. First, a promising extension is to explore counterfactual analysis once a stable DAG and fitted causal model are available, enabling questions such as how an individual job's failure probability might change under alternative resource requests. Second, improving workload characterization would likely strengthen causal identifiability; richer profiling signals or clearer job-intent labels (e.g., separating debug/test from production runs) could help explain the runtime-related patterns we observed. Third, for Theta specifically, investigating non-linear or mixed-type causal discovery and effect estimation methods may help resolve the remaining ambiguity around `EXIT_FAIL`, especially for relationships involving specified walltime. Finally, if future Theta logs include a closer match to the MIT feature set, a careful, constrained cross-system comparison or partial graph transfer study could be revisited in a more principled way.

LLM Use Disclosure

We used generative AI tools (such as ChatGPT) only for limited support during the preparation of this project. Specifically, we used these tools to improve grammar, sentence clarity, and overall fluency in our written report, as well as to help interpret confusing error messages while debugging our own code. All coding, experiment design, data analysis, and causal discovery methodology were fully implemented by the members of our team.

References

- [1] Ju-Won Park, Xin Huang, and Chul-Ho Lee. Analyzing and predicting job failures from hpc system log. *The Journal of Supercomputing*, 80(1):435–462, 2024.
- [2] Xiaoyu Chu, Daniel Hofstätter, Shashikant Ilager, Sacheendra Talluri, Duncan Kampert, Damian Podareanu, Dmitry Duplyakin, Ivona Brandic, and Alexandru Iosup. Generic and ml workloads in an hpc datacenter: Node energy, job failures, and node-job analysis. In *2024 IEEE 30th International Conference on Parallel and Distributed Systems (ICPADS)*, pages 710–719. IEEE, 2024.
- [3] Heonchang YU. Stable hpc cluster management scheme through performance evaluation. In *The 9th International Conference on Ubiquitous Information Technologies and Applications (CUTE 2014)*, pages 1017–1023. KIPS, 2014.
- [4] Lisa compute cluster. <https://www.surf.nl/files/2019-03/lisa-compute-cluster.pdf>, 2019. Accessed: 2025-11-07.
- [5] Siddharth Samsi, Matthew L Weiss, David Bestor, Baolin Li, Michael Jones, Albert Reuther, Daniel Edelman, William Arcand, Chansup Byun, John Holodnack, et al. The mit supercloud dataset. In *2021 IEEE High Performance Extreme Computing Conference (HPEC)*, pages 1–8. IEEE, 2021.
- [6] Alcf theta. https://reports.alcf.anl.gov/data/theta.html#DIM_JOB_COMPOSITE, 2025. Accessed: 2025-11-07.
- [7] Ankur Lahiry, Banooqa Bandy, and Tanzima Z Islam. Wander: An explainable decision-support framework for hpc. *arXiv preprint arXiv:2506.04049*, 2025.
- [8] Pengfei Chen, Yong Qi, Pengfei Zheng, and Di Hou. Causeinfer: Automatic and distributed performance diagnosis with hierarchical causality graph in large distributed systems. In *IEEE INFOCOM 2014-IEEE Conference on Computer Communications*, pages 1887–1895. IEEE, 2014.
- [9] Avinash Maurya, Bogdan Nicolae, Ishan Guliani, and M. Mustafa Rafique. Cosim: A simulator for co-scheduling of batch and on-demand jobs in hpc datacenters. In *2020 IEEE/ACM 24th International Symposium on Distributed Simulation and Real Time Applications (DS-RT)*, pages 1–8, 2020.
- [10] Clark Glymour, Kun Zhang, and Peter Spirtes. Review of causal discovery methods based on graphical models. *Frontiers in genetics*, 10:524, 2019.
- [11] Peter Spirtes. An anytime algorithm for causal inference. In *International Workshop on Artificial Intelligence and Statistics*, pages 278–285. PMLR, 2001.
- [12] Shohei Shimizu, Patrik O Hoyer, Aapo Hyvärinen, Antti Kerminen, and Michael Jordan. A linear non-gaussian acyclic model for causal discovery. *Journal of Machine Learning Research*, 7(10), 2006.
- [13] Patrik Hoyer, Dominik Janzing, Joris M Mooij, Jonas Peters, and Bernhard Schölkopf. Nonlinear causal discovery with additive noise models. *Advances in neural information processing systems*, 21, 2008.
- [14] Xun Zheng, Bryon Aragam, Pradeep K Ravikumar, and Eric P Xing. Dags with no tears: Continuous optimization for structure learning. *Advances in neural information processing systems*, 31, 2018.
- [15] Daniela Schkoda, Philipp Faller, Patrick Blöbaum, and Dominik Janzing. Cross-validating causal discovery via leave-one-variable-out. *arXiv preprint arXiv:2411.05625*, 2024.
- [16] Elias Eulig, Atalanti A Mastakouri, Patrick Blöbaum, Michaela Hardt, and Dominik Janzing. Toward falsifying causal graphs using a permutation-based test. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 26778–26786, 2025.
- [17] Guido W Imbens and Donald B Rubin. *Causal inference in statistics, social, and biomedical sciences*. Cambridge university press, 2015.
- [18] Jonas Peters, Dominik Janzing, and Bernhard Schölkopf. *Elements of causal inference: foundations and learning algorithms*. The MIT press, 2017.
- [19] Patrick Blöbaum, Peter Götz, Kailash Budhathoki, Atalanti A. Mastakouri, and Dominik Janzing. Dowhy-gcm: An extension of dowhy for causal inference in graphical causal models. *Journal of Machine Learning Research*, 25(147):1–7, 2024.
- [20] Zhixuan Chu and Sheng Li. Causal effect estimation: Recent progress, challenges, and opportunities. *Machine Learning for Causal Inference*, pages 79–100, 2023.
- [21] Youfei Yu, Min Zhang, Xu Shi, Megan EV Caram, Roderick JA Little, and Bhramar Mukherjee. A comparison of parametric propensity score-based methods for causal inference with multiple treatments and a binary outcome. *Statistics in Medicine*, 40(7):1653–1677, 2021.
- [22] Christian Hansen, Jerry Hausman, and Whitney Newey. Estimation with many instrumental variables. *Journal of Business & Economic Statistics*, 26(4):398–422, 2008.
- [23] Anthony D Harris, Jessina C McGregor, Eli N Perencevich, Jon P Furuno, Jingkun Zhu, Dan E Peterson, and Joseph Finkelstein. The use and interpretation of quasi-experimental studies in medical informatics. *Journal of the American Medical Informatics Association*, 13(1):16–23, 2006.
- [24] Heining Cham, Hyunjung Lee, and Igor Migunov. Quasi-experimental designs for causal inference: An overview. *Asia Pacific Education Review*, 25(3):611–627, 2024.
- [25] Amit Sharma and Emre Kiciman. Dowhy: An end-to-end library for causal inference. *arXiv preprint arXiv:2011.04216*, 2020.
- [26] Matthew J Vowles, Necati Cihan Camgoz, and Richard Bowden. D'ya like dags? a survey on structure learning and causal discovery. *ACM Computing Surveys*, 55(4):1–36, 2022.
- [27] Elias Eulig, Atalanti A. Mastakouri, Patrick Blöbaum, Michaela Hardt, and Dominik Janzing. Toward falsifying causal graphs using a permutation-based test, 2024.
- [28] Albert Reuther, Jeremy Kepner, Chansup Byun, Siddharth Samsi, William Arcand, David Bestor, Bill Bergeron, Vijay Gadepally, Michael Houle, Matthew Hubbell, et al. Interactive supercomputing on 40,000 cores for machine learning and data analysis. In *2018 IEEE High Performance extreme Computing Conference (HPEC)*, pages 1–6. IEEE, 2018.
- [29] Andy B Yoo, Morris A Jette, and Mark Grondona. Slurm: Simple linux utility for resource management. In *Workshop on job scheduling strategies for parallel processing*, pages 44–60. Springer, 2003.